Task 6: Design Challenge: College Canteen Inventory and Billing (13 Marks)

Stage 1: Interpret the Problem (2 Marks)

Entity	Data
Item (common)	id, name, kind (packaged, cooked or combo), price, stock, units sold, revenue
Packaged item	common data + expiry date
Cooked item	common data + time prepared (hour, minute)
Combo meal	price + up to 3 component item ids (stock is taken from its components)
Date / Time	day, month, year / hour, minute
Sale	item id, quantity (recorded into the item's sold count and revenue)
Canteen	array of items, number of items, today's date

Requirements as testable statements:

ID	Requirement	Test
R1	A sale of quantity q is accepted only if q > 0 and the item exists.	Sell 0 units or id 99: rejected
R2	A sale is rejected if stock < q, and the stock is left unchanged.	Samosa stock 4, sell 10: rejected, stock still 4
R3	After an accepted sale, stock falls by exactly q and revenue rises by q x price.	Sell 5 biscuits at 10: stock 20 to 15, revenue 50
R4	A combo sale needs every component to have stock >= q; either all components are reduced or none are.	3 combos reduce dosa by 3 and juice by 3
R5	A packaged item that expires within 2 days (0 to 2 days from today) triggers a warning; an expired one is flagged and cannot be sold.	Juice expires tomorrow: warning
R6	The summary lists items in descending order of revenue, and the total equals the sum of item revenues.	Order 180, 160, 50, 30, 0; total 420
R7	A combo has 1 to 3 components.	Combo with 4 components not accepted

Stage 2: Propose the Data Model (2 Marks)

Mechanism	Where it is used and why
Nested structures	`struct date` (expiry) and `struct time` (time prepared) are nested in the item; `struct combo_info` (component count and ids) is nested in the union.
Tagged union	An item is exactly one of three kinds. The common fields (id, name, price, stock, sold, revenue) stay outside; the kind-specific data (expiry, prepared time, combo components) share one union, selected by the tag `enum item_kind kind`.
Array of structures	`struct item items[MAX_ITEMS]` inside `struct canteen`, together with the item count and today's date. Items are stored contiguously for easy linear search.
Array of pointers	For the revenue-sorted summary, an array of `struct item *` is sorted instead of moving whole 64-byte items.

Stage 3: Identify the Critical Design Decisions (2 Marks)

- **Safe stock update:** always check quantity and stock first and change stock only afterwards. For combos use two passes: pass 1 validates every component without changing anything; pass 2 deducts. This guarantees no partial deduction, so stock can never become negative. The function modifies the item in place through a pointer (struct item *), not through a copy, and returns a status code (enum sale_status) so that the caller knows why a sale failed.
- **Expiry check with a date structure:** converting a date to a day number (days since a fixed origin) and subtracting gives the exact number of days left, with no mistakes at month ends or leap years. days_until(today, expiry) < 0 means expired; 0 to 2 means warn. Comparing day, month and year fields one by one does not give "within two days".
- **Static or global data:** the canteen object is large and must start empty, so it is a static object (zero-initialised), created in one place and passed by pointer everywhere. The helper functions day_number and days_until are static (file-private). The status-message table is a static const array. MAX_ITEMS and WARN_DAYS are constants (#define). Today's date is kept in the canteen structure rather than a mutable global, so it can be changed for testing. No other global variable is used.

Stage 4: Design the Structures and Functions (4 Marks)

Structure definitions

```
#define MAX_ITEMS 20
#define WARN_DAYS 2

struct date { int day, month, year; };
struct time { int hour, minute; };

enum item_kind { PACKAGED, COOKED, COMBO };
enum sale_status { SALE_OK, SALE_NO_ITEM, SALE_BAD_QTY, SALE_NO_STOCK, SALE_EXPIRED };

struct combo_info { int count; int item_id[3]; };

struct item {
   int   id;
   char  name[24];
   enum item_kind kind;              /* tag */
   float price;
   int   stock;                      /* unused for COMBO */
   int   sold;
   float revenue;
   union {
      struct date       expiry;      /* PACKAGED */
      struct time       prepared;    /* COOKED   */
      struct combo_info combo;       /* COMBO    */
   } u;
};

struct canteen {
   struct item items[MAX_ITEMS];
   int count;
   struct date today;
};
```

Function prototypes (all use structure pointers)

```
struct item     *find_item(struct canteen *c, int id);
int             add_item(struct canteen *c, const struct item *it);
enum sale_status record_sale(struct canteen *c, int id, int qty);
void            expiry_warning(const struct canteen *c);
void            bill_summary(struct canteen *c);
static long     day_number(const struct date *d);
static int      days_until(const struct date *from, const struct date *to);
```

Pseudocode: record a sale

```
RECORD_SALE(c, id, qty)
  it <- FIND_ITEM(c, id)
  if it = NULL        then return NO_ITEM
```

```
  if qty <= 0          then return BAD_QTY
  if it.kind = COMBO then
     for each component k of it                  // pass 1: validate only
        part <- FIND_ITEM(c, k)
        if part = NULL or part.stock < qty then return NO_STOCK
        if part is PACKAGED and DAYS_UNTIL(today, part.expiry) < 0
                                          then return EXPIRED
     for each component k of it                  // pass 2: commit
        FIND_ITEM(c, k).stock <- stock - qty
  else
     if it.stock < qty                    then return NO_STOCK
     if it is PACKAGED and DAYS_UNTIL(today, it.expiry) < 0
                                       then return EXPIRED
     it.stock <- it.stock - qty
  it.sold    <- it.sold + qty
  it.revenue <- it.revenue + qty * it.price
  return OK
```

Pseudocode: expiry warning

```
EXPIRY_WARNING(c)
  any <- false
  for each item it in c.items
     if it.kind != PACKAGED then continue
     left <- DAYS_UNTIL(c.today, it.expiry)       // via day numbers
     if left < 0           then print "EXPIRED", it.name ; any <- true
     else if left <= 2     then print "WARNING", it.name, left ; any <- true
  if not any then print "No items expiring within 2 days"
```

Stage 5: Implement and Demonstrate (3 Marks)

Full program (compiled and run). It demonstrates normal sales, a sale that exceeds stock, an expiring item and the sorted summary. Today is fixed at 12/11/2026.

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 20
#define WARN_DAYS 2

struct date { int day, month, year; };
struct time { int hour, minute; };

enum item_kind { PACKAGED, COOKED, COMBO };
enum sale_status { SALE_OK, SALE_NO_ITEM, SALE_BAD_QTY, SALE_NO_STOCK, SALE_EXPIRED };

struct combo_info { int count; int item_id[3]; };

struct item {
    int   id;
    char  name[24];
    enum item_kind kind;                /* tag */
    float price;
    int   stock;                        /* unused for COMBO */
    int   sold;
    float revenue;
    union {
        struct date       expiry;       /* PACKAGED */
        struct time       prepared;     /* COOKED   */
        struct combo_info combo;        /* COMBO    */
    } u;
};

struct canteen {
    struct item items[MAX_ITEMS];
    int count;
    struct date today;
};

/* ---------- date helper: days since 1970-01-01 ---------- */
static long day_number(const struct date *d)
{
```

```
    int y = d->year, m = d->month;
    if (m <= 2) { y--; m += 12; }
    return 365L * y + y / 4 - y / 100 + y / 400 + (153 * (m - 3) + 2) / 5 + d->day;
}

static int days_until(const struct date *from, const struct date *to)
{
    return (int)(day_number(to) - day_number(from));
}

/* ---------- operations ---------- */
struct item *find_item(struct canteen *c, int id)
{
    for (int i = 0; i < c->count; i++)
        if (c->items[i].id == id) return &c->items[i];
    return NULL;
}

int add_item(struct canteen *c, const struct item *it)
{
    if (c->count >= MAX_ITEMS) return 0;
    c->items[c->count++] = *it;
    return 1;
}

enum sale_status record_sale(struct canteen *c, int id, int qty)
{
    struct item *it = find_item(c, id);
    if (it == NULL)  return SALE_NO_ITEM;
    if (qty <= 0)    return SALE_BAD_QTY;

    if (it->kind == COMBO) {
        /* PASS 1: validate every component, change nothing */
        for (int k = 0; k < it->u.combo.count; k++) {
            struct item *part = find_item(c, it->u.combo.item_id[k]);
            if (part == NULL || part->stock < qty) return SALE_NO_STOCK;
            if (part->kind == PACKAGED && days_until(&c->today, &part->u.expiry) < 0)
                return SALE_EXPIRED;
        }
        /* PASS 2: commit */
        for (int k = 0; k < it->u.combo.count; k++)
            find_item(c, it->u.combo.item_id[k])->stock -= qty;
    } else {
        if (it->stock < qty) return SALE_NO_STOCK;
        if (it->kind == PACKAGED && days_until(&c->today, &it->u.expiry) < 0)
            return SALE_EXPIRED;
        it->stock -= qty;
    }
    it->sold    += qty;
    it->revenue += qty * it->price;
    return SALE_OK;
}

void expiry_warning(const struct canteen *c)
{
    int any = 0;
    for (int i = 0; i < c->count; i++) {
        const struct item *it = &c->items[i];
        if (it->kind != PACKAGED) continue;
        int left = days_until(&c->today, &it->u.expiry);
        if (left < 0)
            { printf("  EXPIRED : %-16s (expired %d day(s) ago)\n", it->name, -left); any = 1; }
        else if (left <= WARN_DAYS)
            { printf("  WARNING : %-16s expires in %d day(s)\n", it->name, left); any = 1; }
    }
    if (!any) printf("  No items expiring within %d days.\n", WARN_DAYS);
}

void bill_summary(struct canteen *c)
{
    struct item *order[MAX_ITEMS];          /* sort pointers, not structures */
    for (int i = 0; i < c->count; i++) order[i] = &c->items[i];
```

```
    for (int i = 0; i < c->count - 1; i++) {   /* selection sort, descending */
        int max = i;
        for (int j = i + 1; j < c->count; j++)
            if (order[j]->revenue > order[max]->revenue) max = j;
        struct item *t = order[i]; order[i] = order[max]; order[max] = t;
    }
    float total = 0;
    printf("  %-16s %5s %9s\n", "Item", "Sold", "Revenue");
    for (int i = 0; i < c->count; i++) {
        printf("  %-16s %5d %9.2f\n", order[i]->name, order[i]->sold, order[i]->revenue);
        total += order[i]->revenue;
    }
    printf("  %-16s %5s %9.2f\n", "TOTAL", "", total);
}

static void report(const char *what, enum sale_status s)
{
    static const char *msg[] = {"OK", "item not found", "bad quantity",
                                "REJECTED - insufficient stock", "REJECTED - item expired"};
    printf("  %-26s -> %s\n", what, msg[s]);
}

int main(void)
{
    static struct canteen c = { .count = 0, .today = {12, 11, 2026} };

    struct item biscuit = {1, "Biscuit pack", PACKAGED, 10.0f, 20, 0, 0, .u.expiry = {30, 11, 2026}};
    struct item juice   = {2, "Juice box",    PACKAGED, 25.0f, 10, 0, 0, .u.expiry = {13, 11, 2026}};
    struct item dosa    = {3, "Masala dosa",  COOKED,   40.0f, 15, 0, 0, .u.prepared = {8, 30}};
    struct item samosa  = {4, "Samosa",       COOKED,   15.0f,  6, 0, 0, .u.prepared = {9, 15}};
    struct item combo   = {5, "Dosa+Juice",   COMBO,    60.0f,  0, 0, 0, .u.combo = {2, {3, 2}}};
    add_item(&c, &biscuit); add_item(&c, &juice); add_item(&c, &dosa);
    add_item(&c, &samosa);  add_item(&c, &combo);

    printf("TEST 1: normal sales\n");
    report("sell 5 biscuit packs",  record_sale(&c, 1, 5));
    report("sell 4 masala dosa",    record_sale(&c, 3, 4));
    report("sell 2 samosa",         record_sale(&c, 4, 2));
    report("sell 3 combos",         record_sale(&c, 5, 3));

    printf("\nTEST 2: sale exceeding stock\n");
    report("sell 10 samosa (stock 4)", record_sale(&c, 4, 10));
    printf("  samosa stock still = %d (unchanged)\n", find_item(&c, 4)->stock);

    printf("\nTEST 3: expiry warning (today = 12/11/2026)\n");
    expiry_warning(&c);

    printf("\nTEST 4: sorted bill summary (highest revenue first)\n");
    bill_summary(&c);
    return 0;
}
```

Output

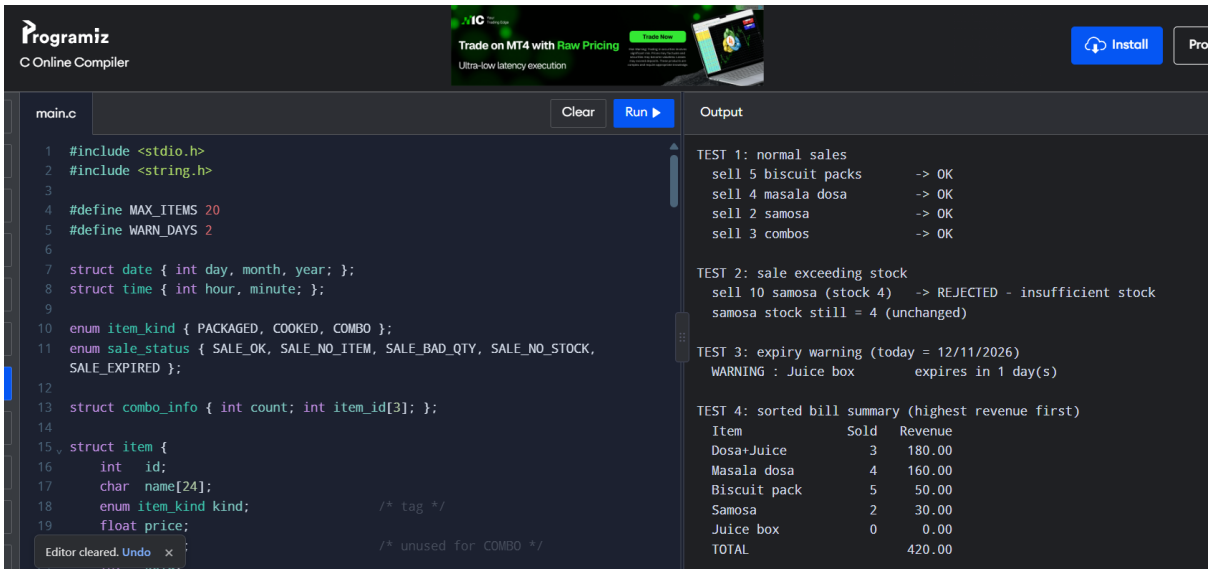
```
TEST 1: normal sales
  sell 5 biscuit packs       -> OK
  sell 4 masala dosa         -> OK
  sell 2 samosa              -> OK
  sell 3 combos              -> OK

TEST 2: sale exceeding stock
  sell 10 samosa (stock 4)   -> REJECTED - insufficient stock
  samosa stock still = 4 (unchanged)

TEST 3: expiry warning (today = 12/11/2026)
  WARNING : Juice box        expires in 1 day(s)

TEST 4: sorted bill summary (highest revenue first)
  Item             Sold   Revenue
  Dosa+Juice          3    180.00
  Masala dosa         4    160.00
  Biscuit pack        5     50.00
  Samosa              2     30.00
```

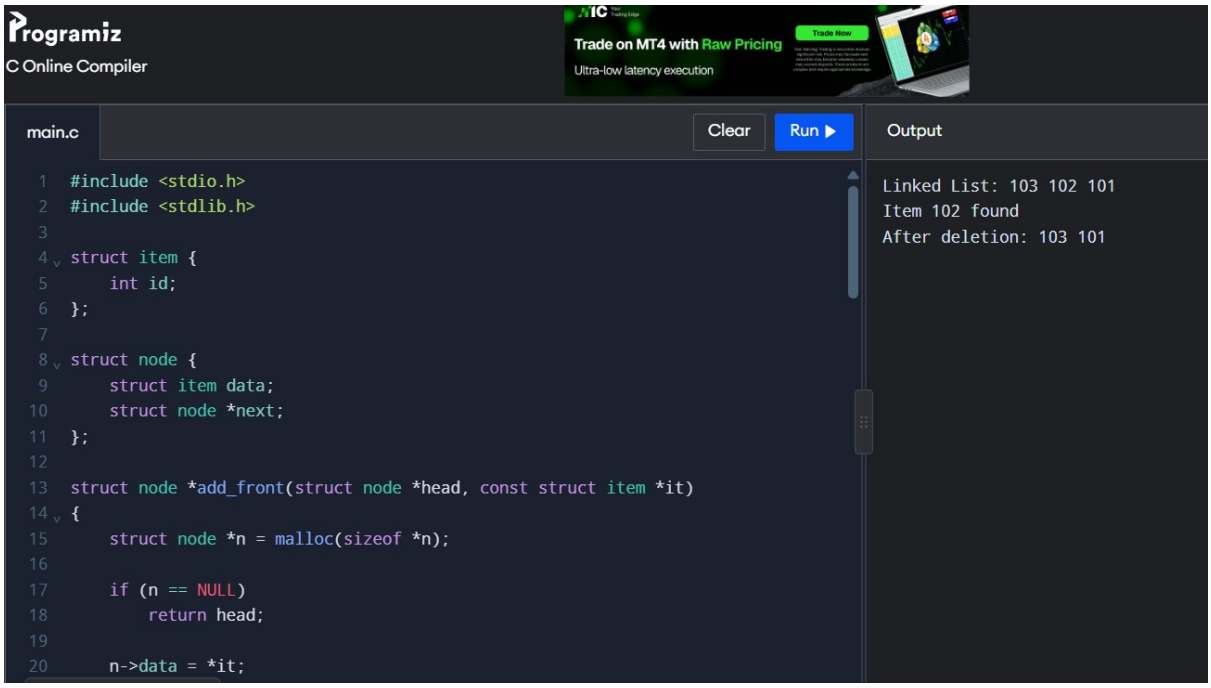
```
Juice box       0     0.00
TOTAL                 420.00
```



Alternative Concept and Complexity Analysis: Linked List

```
struct node {
    struct item  data;
    struct node *next;      /* extra pointer: 8 bytes */
};

struct node *add_front(struct node *head, const struct item *it) {
    struct node *n = malloc(sizeof *n);
    if (!n) return head;
    n->data = *it;  n->next = head;
    return n;                           /* O(1) */
}
struct node *find(struct node *h, int id) {     /* O(n) */
    for (; h; h = h->next) if (h->data.id == id) return h;
    return NULL;
}
struct node *remove_id(struct node *h, int id) { /* O(n) search + O(1) unlink */
    struct node *prev = NULL, *cur = h;
    while (cur && cur->data.id != id) { prev = cur; cur = cur->next; }
    if (!cur) return h;
    if (prev) prev->next = cur->next; else h = cur->next;
    free(cur);
    return h;
}
```



	Array of structures	Linked list of malloc'd structures
Add	O(1) at the end (if capacity remains); fails when MAX_ITEMS is reached	O(1) at the head (O(1) at the tail with a tail pointer; O(n) without). No capacity limit, but one malloc per item
Remove	O(n): find O(n), then shift up to n-1 elements of 64 bytes each	O(n) to find, then O(1) to unlink; no shifting; free the node
Search	O(n) linear; O(log n) with binary search if kept sorted by id	O(n); binary search is impossible
Extra space	None per item, but unused capacity is wasted: (MAX_ITEMS - n) x 64 bytes	One next pointer per item: 8n bytes, plus malloc bookkeeping (about 8 to 16 bytes per node)
Memory behaviour	Contiguous: excellent cache behaviour, no allocation failures after start	Scattered nodes: more cache misses; must check malloc failure and avoid leaks

Evaluation. For a canteen with a small, nearly fixed menu (about 20 items), the array is simpler and faster. A linked list is better only if the number of items changes frequently and has no sensible upper limit.

Overall Feedback and Conclusion

This assignment improved my understanding of structures, unions, pointers, storage classes and memory management in C. It helped me develop problem-solving, debugging and analytical skills. The practical programs and complexity analysis provided valuable experience in designing efficient and reliable data structures for real-world applications.